

Assignment 1 Submission

Nopparuj Sodsri and Supanutt Maleemongkol

CS3210 – 2025/26 Semester 2

1 Implementation

1.1 Overview

The simulator is represented by an array of cars provided by skeleton code. Each Car has a data of id, lane, position, and speed. We partition the array into 2 sub-array with different lane numbers and sort each sub-array by car's position, and we implement sorting to maintain this order. For more details on data structure, see Section 1.2. For each timestep, each car is checked whether it satisfies all the conditions for lane switching. Then, we call `flip_coin()` twice. One call for each car to decide if they will accelerate normally, and another to for random deceleration. Then, cars' velocities are updated accordingly to Nagel-Schrekenberg + Extension 1, where velocity of the i -th car and its nearest next car in the previous step are used to check conditions in steps 1-4 and lane switching. For lane switching, nearest next car in different is done by binary search (see section 1.2).

At the end of each timestep, car velocities are updated and their position are updated accordingly. At the end of simulation, the array is sorted based on to give the correctly ordered answer.

1.2 Data Structure

All cars are kept and sorted in an array of size n , the number of cars for the simulation. We keep track of an integer `splitIndex`, where the cars from index 0 to the index (`splitIndex - 1`) is in lane 0 and the cars from index (`splitIndex`) to index $n - 1$ is in lane 1. All the cars in the same lane is sorted by their position on the road in ascending order. For this data structure we have a `nextCar()` and `prevCar()` method that returns an iterator to the next car or previous car in the same/different lane in worst case $O(\log(n))$ time. We also utilized a vector of `uint8_t` to keep track of multiple flags for each car. These include whether or not they are skipping certain step when updating position or whether they need to start in the next timestep. `uint8_t` was use instead of boolean, since writing requires updating of the whole byte, and in parallel there is a race condition.

1.3 Parallelization Strategy

For each iteration, We use `#pragma omp parallel` for to spawn threads pool and to divide arrays of car to split the work. The cars array is implicitly a *shared* memory. This is necessary, especially when we need to find the nearest next car in either same or different lane. Furthermore, we split the code into three parts that require synchronization: lane switching, RNG generation, and velocity/position update. We parallelize each part separately.

The first part is lane switching. lane switching in each timestep used the information from the previous timestep, each car can be swap independently by referring to the previous timestep information and updating the cars lane into a temporary array, the reading and writing here does not cause a race condition. The temporary array's information is then transfer to cars array using `std::swap()`, which only requires $O(1)$ time complexity.

The second part RNG step (calling `flip_coin()`) is more difficult to parallelize because the output depends on call order. We cannot share a single PRNG across threads, since the call order is nondeterministic. Instead, we split the n calls into x contiguous groups, where x is the number of threads. Using `#pragma omp single`, we compute the per-thread workload, then synchronize with `#pragma omp barrier`. Each thread creates a local PRNG copy and, using its `thread_id`, advances (discards) the PRNG state to the correct starting point for its assigned range of `cars_id`,

then runs `flip_coin()` for that range. Finally, the main PRNG discards $2*n$ values to advance by the same total number of calls for the timestep, ensuring the next timestep starts at the correct point. This preserves correctness relative to the sequential code.

After determining which cars need to delay their starts or decelerate, we update each car velocity in step 1-4 and position in step 5 accordingly. This is simple to parallelize: threads read the results from the first two stages and write updates to a temporary array, so no race condition occurs. The only shared values are `rotate0` and `rotate1`, which track how far to rotate each lane after the position update. We handle this with `reduction(+:rotate0, rotate1)` to solve the race condition when updating these values. `reduction` directive is used to combine each thread's private partial sums into a single shared result without race conditions, optimizing parallelism.

Sorting method is mergesort, which is known to be compatible for parallelization. We used `task` and `taskwait` directives of OpenMP. Unlike sections, tasks allow threads to execute recursive subproblems asynchronously. Additionally, we used `#pragma omp taskgroup` so that both recursive `mergeSort()` calls must be finished before preceding `merge()`.

2 Optimization

Under quick investigation, we found that our main performance bottleneck is due to `mergeSort` function. This is mainly due to time complexity $O(n \log n)$ for each time step. Nevertheless, there are multiple aspects we could improve performance of `mergeSort`.

2.1 Ping-Pong Buffer in `mergeSort`

Instead of one copy per `merge` call, array of interest is also used as a buffer, swapping role with temporary array at next recursion level. Therefore, you only need to copy whole array once at the beginning. This should significantly decrease the number of loads and load misses.

2.2 `insertionSort` for small array

For a small number of cars $n < 1000$, `insertionSort` is preferred since it has better cache performance, and is particularly effective for the cases where the array is almost ordered, for example, there are very few lane switching, where it could be $O(n)$ for $O(1)$ lane switching. For this reason, we added the condition to tell `\mergeSort()` to do just serial `insertionSort()` if the size is less than 1024.

2.3 Reduced Synchronization: Combining Loops

Initially, our code contains many for-loops: lane switching, RNG generation for p_{dec} and p_{start} , copying the cars array into a temporary array, updating velocity, and updating position. Parallelizing each loop separately with `#pragma omp parallel` for introduced significant overhead, both from repeatedly creating threads and from synchronizing after every loop. To address the problem, we decided to combine sections of code where the data are not interdependent, specifically the velocity and position updates. Since each car updates its own position based on its own (updated) velocity, and velocity updates depend only on values from the pre-update state rather than on other threads' writes, there is no race condition.

2.4 Other Improvement

Here are a few other improvements we did:

- condition to skip the sorting completely if there is no lane switching in that time step.
- rotating the array after car position update instead of sorting: $O(n)$ instead of $O(n \log n)$

3 Performance Analysis

3.1 Performance indicator and General Methodology

These are parameters we considered for the performance of the program

- execution time (duration elapse)
- LLC loads and load misses
- speed up

For each test, we repeated 10 times for a more reliable result (5 times for thread analysis, Section 4). The reproduction guideline can be viewed in section 5.2.

3.2 Simulation Parameter

Some parameters have more effect to performance than others. We selected some of them that are expected to be the most impactful to program efficiency as a variable. These include

- number of cars n : higher cost for merging and finding next car in different lane
- density of cars, roughly reflected by L/n
- p_{dec} and p_{start} : affect branching prediction and memory access pattern

For this part, we will use only use i7-13700 CPU.

3.2.1 Number of Cars and Car Sparsity

For the first part of our performance analysis, we vary the road length L relative to the number of cars n . We will be calling this parameter road sparsity. Specifically, we measure many performance measurements for running the traffic simulation program with varying numbers of cars n , from 10000 to 10000000 cars with the road length of $10n$, $20n$, $50n$, and $100n$.

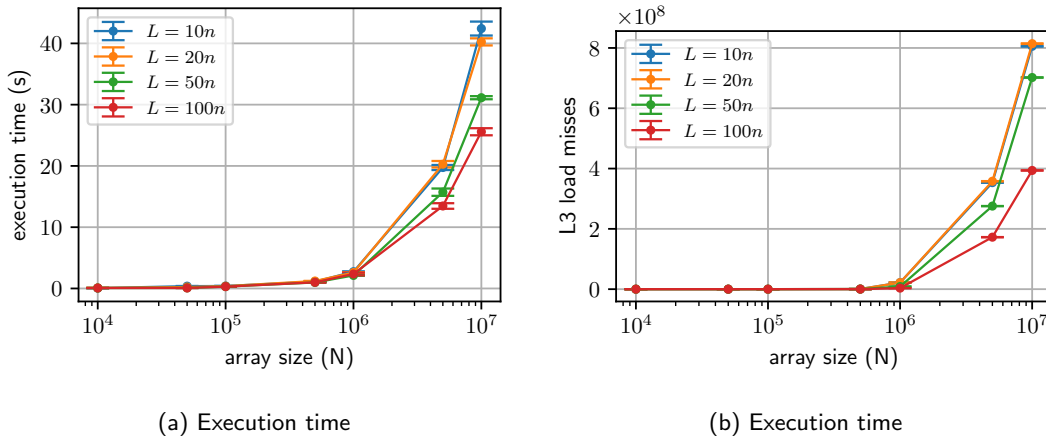


Figure 1: Performance scaling on number of cars and car densities

According to Figure 1, it can be seen that as the road's traffic becomes more sparsed, the number of cycles and execution time go down. This is more apparent for higher n values. A possible explanation for this is that sparser traffic means cars are less likely to be close enough for specific conditions to be fulfilled, such as avoiding crashes and switching lanes. Hence, there is a higher chance that zero lane switchint occurs in a time step, allowing the program to completely skip merging, as done by optimization (See 2.4).

Another observation we have is that the cache miss rate across different sparsity levels tends to peak at around 50,000 and 100,000 cars. We believe that this is because the CPU can predict

which memory locations will need to be accessed next when the n gets large, as a greater proportion of time is spent in each region of the code instead of switching between regions of the code. For instance, as the number of cars becomes larger, a bigger proportion of the execution time for portion of the code like copying the array, which is just reading memory in order, which is easier for the CPU to predict.

3.2.2 p_{dec} and p_{start}

For the second part of our performance analysis, we test the simulation by varying the probability p_{start} and p_{dec} . The goal is to see how randomness may affect the program performance.

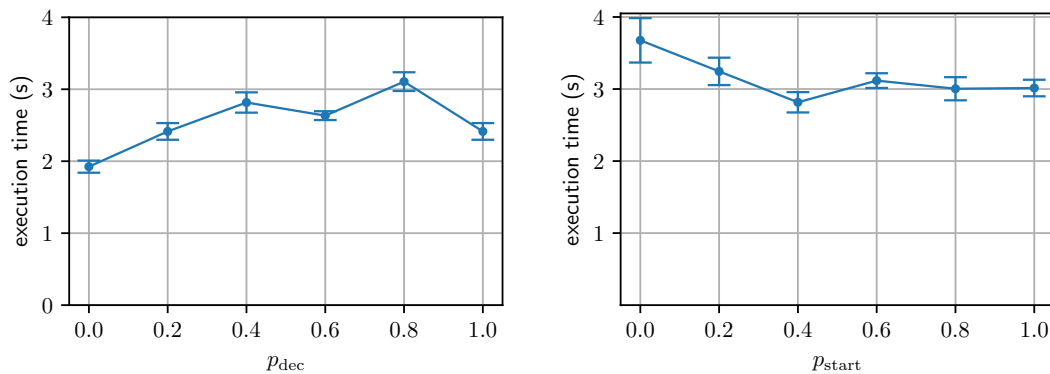


Figure 2: Performance on different value of p_{dec} and p_{start}

From Figure 2 when we fix p_{start} and adjust p_{dec} , we can see that the execution time is lower when the probability of p_{dec} is set to 0 and 1.0. This is likely because both produce a more consistent traffic behavior, so the simulator executes fewer conditions, which is why the instruction counts are half of those of other values. Cache miss rate being one-fourth of other values suggests that the memory access pattern became quite predictable, so the CPU is able to prefetch the necessary memory.

When we fix p_{dec} and vary the value of p_{start} , the changes is less noticeable than when p_{dec} was changing. All measurements are pretty stable between changing p_{start} values. However, there seems to be a small decreasing for instruction count for each incremental increase in p_{start} . This could be explained by the simulation dealing with the slow start condition less frequently, since slow start requires more instruction than just starting regularly.

3.3 Performance on Different CPU Models

The objective of this benchmark is to compare how different CPUs architectures handle this simulation and how the CPU's frequency, number of threads, and cache size could affect its performance. We use 4 models of CPU for this analysis: the i7-13700, i7-7700, w5-3423, and the xs-4114.

3.3.1 Number of Cars

Across all four machines, execution time increases with the number of cars as expected. For small inputs, miss rates are negligible on all CPUs, suggesting that nearly all the necessary memory accesses were able to nicely fit within the cache. For all car sizes, the i7-13700 tends to have the fastest execution time, while the other three are quite similar. i7-13700 being the most advanced CPU here with a higher frequency and more advanced design could be used to explain why it was able to outperform the rest of the CPUs. One interesting observation is that the miss rate for the

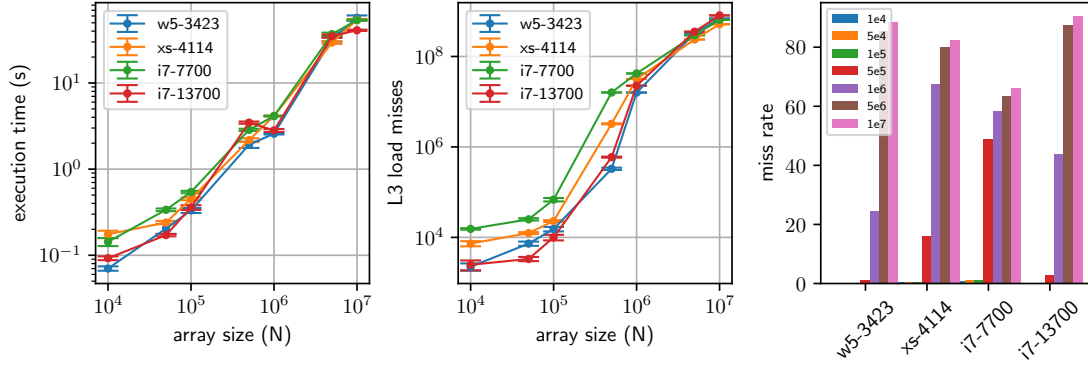


Figure 3: Performance of Different CPU on scaling number of cars: execution time (left), L3 load misses (middle), L3 load miss rate (right)

i7-7700 CPU is much lower than the other three CPU for $n = 10000000$, despite being one the cheapest and most outdated CPU here. One possible explanation is that the i7-7700 has only 8 thread, compared to i7-13700's 24 threads, w5-3423's 24 threads, and xs-4114's 20 threads. Having fewer threads could mean less cache contention, therefore would explain the lower proportion of cache misses observed.

3.3.2 Number of Threads

From Figure 4, as we adjust the number of threads on each machine, we can see that the execution time decreases at a decreasing rate. In this context, the number of thread = 1 refers to the sequential code being run on that machine. We observe that speedup more than double when the number of threads double from one to two, showing superlinear speedup. However as thread counts continue to increase, this cache benefit that allow superlinear scaling is eventually outweighed by contention and overhead, leading to diminishing returns. Note that i7-7700 doesn't have enough processing unit to run on 16 threads, so it is left blank in the graph at number of threads of 16.

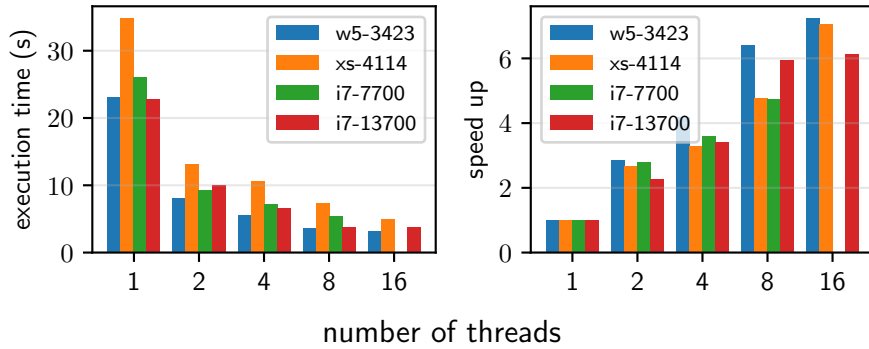


Figure 4: Time Execution and Speed up on different CPU Models ($N = 1e6$)

4 Conclusion

This project implements a modified two-lane NS traffic simulator and improves execution time through algorithmic optimizations and OpenMP parallelization. Performance benchmarks were run to see how different parameters affect performance and how different CPU architectures behave differently under the same task.

5 Appendix

Please refer to [GitHub](#), [GitHub](#), and [GitHub](#).

5.1 Raw Code

A raw code can be found in [GitHub](#).

5.2 Result Reproduction Guideline

With desired input, inside the root directory, run `./perf.sh` to test the performance of `sim.perf`. The following is the content of `.perf.sh`:

```
#!/bin/bash
#SBATCH --job-name=myjob
#SBATCH --nodes=1
#SBATCH --ntasks=1
#SBATCH --mem=1gb
#SBATCH --time=00:20:00
#SBATCH --output=log/%j.log
#SBATCH --error=log/%j.log
##SBATCH --partition=i7-7700

INPUT=$1
NUM_THREADS=${2:-${SLURM_CPUS_PER_TASK:-16}}
export OMP_NUM_THREADS=$NUM_THREADS

mkdir -p log
mkdir -p perf_report

echo " Running job !"
echo "We are running on $( hostname )"
echo "Job started at $( date )"

divider=$(printf '=%.0s' {1..50})
echo "$divider"
echo "USER EXECUTABLE PERFORMANCE STATISTICS"
echo "input file: $INPUT"
echo "partition: ${SLURM_JOB_PARTITION}"
if [ -n "${SLURM_CPUS_PER_TASK}" ]; then
echo "number of threads: $NUM_THREADS"
fi
echo "$divider"

# 5 iteration of threads
srun perf stat -r 10 -e user_time,cycles,instructions,\
    LLC-loads,LLC-load-misses -- bash -c "./sim.perf < $INPUT > /dev/null"
#srun perf record -B -N -e instructions,cycles, \
#    LLC-load-misses -o "perf_report/${SLURM_JOB_ID}_report.data" \
#    -- ./sim.perf < $INPUT > /dev/null
echo "Job ended at $( date )"
```

`perf.sh` has 1 required input and 1 optional input:

- input file: required
- number of thread: optional. If it is specified, then `NUM_OMP_THREADS` will be set.

To select partition, use `-p` flag after `sbtach`, followed by partition name. After obtaining result in the log folder, run `./makedata.sh` scripts. This scripts will search for all and write all output file. We used google sheet with uses of filter to find.

In addition, `bench-4_perf.sh` and `bench-5_perf.sh` are provided to test performance against our code :)

5.3 Parameters for Testing

The input parameter will be fixed at the following value unless specified otherwise:

- number of cars $n = 1000000$:
- road length $n = 10000000$:
- $p_{start} = 0.5$
- $p_{dec} = 0.5$
- time step $t = 100$
- $v_{max} = 100$

5.4 Raw Data

Raw data can be found in `/csv` folder inside [GitHub](#). Alternatively, `/raw-data` folder contains direct output of which is scanned by `./makedata.sh` to generate `.csv` file.

5.5 Input Generation

we use seed 6767 throughout performance analysis. Here is an example.

```
python3 gen.py \  
  --n 10000 \  
  --L 100000 \  
  --vmax 100 \  
  --p-dec 0.5 \  
  --p-start 0.5 \  
  --steps 100 \  
  --seed 6767 \  
  --out performanceReport/cpuModel/1e4
```

5.6 Total Hour Works

67++ hours